Dealer WMS Core System Specification

Master Architecture, Database Schema, API Contracts & Implementation Blueprint

Target Phase: Phase 1 (Dealer Core)	**Tech Stack:** React + NestJS + Postgres	**ORM / DB:** Prisma ORM / PostgreSQL 15+	**Document Ver:** 1.0.0-PROD
Multi-Tenancy: Row-Level Isolation (`dealer_id`)	**Auth Mechanism:** JWT + Argon2id Hash	**Concurrency:** DB Row Locks (`FOR UPDATE`)	**Status:** Complete & Approved

1. Executive Summary & System Scope

This master specification establishes the complete technical and operational blueprint for the **Dealer WMS Core System**. The application is designed specifically for wholesale product distributors (Dealers) who operate as the vital intermediary between product manufacturers and retail shops.

> **Core Business Scope of Phase 1**
>
> The Phase 1 release focuses entirely on delivering a high-performance, secure, multi-tenant warehouse management engine for Dealers. It digitizes the complete lifecycle: issuing Purchase Orders to Manufacturers, receiving bulk goods at loading docks (GRN), managing multi-warehouse stock allocations, enforcing credit limits on retail shops, and picking, packing, and dispatching Sales Orders to retail shops.

Key Operational Capabilities

- **Multi-Tenant Data Isolation:** Complete logical data partitioning using `dealer_id` foreign keys enforced at the NestJS interceptor layer.
- **Inbound Procurement (Procure-to-Pay):** Purchase Order management, Goods Receipt Notes (GRN), backorders, and automated supplier catalog indexing.
- **Real-Time Stock Engine:** Dynamic calculation of Available Stock vs. Allocated Stock with explicit concurrency protection against double-selling.
- **Outbound Fulfillment (Order-to-Cash):** Sales Order placement, automated credit limit checks, picking/packing list generation, dispatching, and proof of delivery.
- **Immutable Movement Ledger:** Strict append-only logging of every stock modification (`stock_movements`) for zero-discrepancy financial and inventory auditing.

2. System Architecture & Multi-Tenancy Strategy

The platform follows a **Modular Monolith Architecture** using NestJS on the backend and React (TypeScript) on the frontend. Multi-tenancy is implemented via **Shared Database, Shared Schema** with strict row-level isolation (`dealer_id`).

2.1 JWT Context & Request Execution Flow

When an employee logs in, the system generates an encrypted JWT token containing their `userId`, `dealerId`, and assigned `role`. Every subsequent API request carries this token in the `Authorization: Bearer ` header.

```
[Client Request: GET /api/v1/inventory]
       |
       ▼
[NestJS JwtAuthGuard] ──> Validates JWT Signature & Expiry
       |
       ▼
[NestJS TenantInterceptor] ──> Extracts `dealerId` from JWT Payload & Binds to Execution Request Context
       |
       ▼
[Prisma Middleware / Service] ──> Appends `WHERE dealer_id = context.dealerId` to DB Query
       |
       ▼
[PostgreSQL DB] ──> Returns strictly isolated tenant data
```

2.2 Role-Based Access Control (RBAC) Matrix

Role	System Permissions & Capabilities	Target User Persona
ADMIN	Full CRUD over tenant users, company settings, financial reports, credit limit overrides, warehouses, and master catalogs.	Dealer Company Owner, Director
MANAGER	Approves POs and SOs, triggers manual stock adjustments, overrides inventory counts, creates products and shop profiles.	Warehouse Operations Manager
STAFF	Performs stock receiving (GRN) at docks, updates pick/pack order statuses, views inventory counts, logs movements.	Warehouse Dock Worker, Picker
DRIVER	Read-only access to assigned delivery routes, updates dispatch state (`SHIPPED` → `DELIVERED`), captures shop signatures.	Delivery Driver

3. Complete Database Schema (PostgreSQL DDL)

Below is the relational PostgreSQL schema definitions including check constraints, foreign keys, and indexes.

```sql
-- 1. TENANTS & USERS
CREATE TABLE dealers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    tax_id VARCHAR(100),
    status VARCHAR(50) DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE', 'SUSPENDED', 'INACTIVE')),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
    email VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    role VARCHAR(50) NOT NULL CHECK (role IN ('ADMIN', 'MANAGER', 'STAFF', 'DRIVER')),
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(dealer_id, email)
);

-- 2. CRM (SUPPLIERS & CUSTOMERS)
CREATE TABLE manufacturers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    contact_name VARCHAR(255),
    email VARCHAR(255),
    phone VARCHAR(50),
    lead_time_days INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE shops (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    contact_name VARCHAR(255),
    email VARCHAR(255),
    phone VARCHAR(50),
    shipping_address TEXT NOT NULL,
    billing_address TEXT,
    credit_limit NUMERIC(12, 2) DEFAULT 0.00,
    current_balance NUMERIC(12, 2) DEFAULT 0.00,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 3. INVENTORY & CATALOG
CREATE TABLE warehouses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    code VARCHAR(50) NOT NULL,
    address TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(dealer_id, code)
);

CREATE TABLE categories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    parent_id UUID REFERENCES categories(id) ON DELETE SET NULL
);
```

```
CREATE TABLE products (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
    manufacturer_id UUID REFERENCES manufacturers(id),
    category_id UUID REFERENCES categories(id),
    sku VARCHAR(100) NOT NULL,
    name VARCHAR(255) NOT NULL,
    barcode VARCHAR(100),
    cost_price NUMERIC(12, 2) NOT NULL CHECK (cost_price >= 0),
    selling_price NUMERIC(12, 2) NOT NULL CHECK (selling_price >= 0),
    reorder_level INT DEFAULT 10,
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(dealer_id, sku)
);

CREATE TABLE inventory (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    warehouse_id UUID NOT NULL REFERENCES warehouses(id) ON DELETE CASCADE,
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    quantity_on_hand INT NOT NULL DEFAULT 0 CHECK (quantity_on_hand >= 0),
    quantity_allocated INT NOT NULL DEFAULT 0 CHECK (quantity_allocated >= 0),
    updated_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(warehouse_id, product_id)
);

CREATE TABLE stock_movements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
    warehouse_id UUID NOT NULL REFERENCES warehouses(id),
    product_id UUID NOT NULL REFERENCES products(id),
    user_id UUID REFERENCES users(id),
    movement_type VARCHAR(50) NOT NULL CHECK (movement_type IN ('PO_RECEIPT', 'SO_DISPATCH', 'ADJUSTMENT',
'TRANSFER_IN', 'TRANSFER_OUT')),
    quantity_change INT NOT NULL,
    reference_id UUID,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 4. PROCUREMENT & FULFILLMENT ORDERS
CREATE TABLE purchase_orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
    manufacturer_id UUID NOT NULL REFERENCES manufacturers(id),
    warehouse_id UUID NOT NULL REFERENCES warehouses(id),
    po_number VARCHAR(50) NOT NULL,
    status VARCHAR(50) DEFAULT 'DRAFT' CHECK (status IN ('DRAFT', 'ORDERED', 'PARTIAL', 'COMPLETED',
'CANCELLED')),
    total_amount NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(dealer_id, po_number)
);

CREATE TABLE purchase_order_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    po_id UUID NOT NULL REFERENCES purchase_orders(id) ON DELETE CASCADE,
    product_id UUID NOT NULL REFERENCES products(id),
    quantity_ordered INT NOT NULL CHECK (quantity_ordered > 0),
    quantity_received INT NOT NULL DEFAULT 0 CHECK (quantity_received >= 0),
    unit_cost NUMERIC(12, 2) NOT NULL
);

CREATE TABLE sales_orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
    shop_id UUID NOT NULL REFERENCES shops(id),
    warehouse_id UUID NOT NULL REFERENCES warehouses(id),
```

```
    so_number VARCHAR(50) NOT NULL,
    status VARCHAR(50) DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'APPROVED', 'PICKING', 'SHIPPED',
'DELIVERED', 'CANCELLED')),
    total_amount NUMERIC(12, 2) NOT NULL DEFAULT 0.00,
    assigned_driver_id UUID REFERENCES users(id),
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(dealer_id, so_number)
);

CREATE TABLE sales_order_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    so_id UUID NOT NULL REFERENCES sales_orders(id) ON DELETE CASCADE,
    product_id UUID NOT NULL REFERENCES products(id),
    quantity_ordered INT NOT NULL CHECK (quantity_ordered > 0),
    quantity_fulfilled INT NOT NULL DEFAULT 0 CHECK (quantity_fulfilled >= 0),
    unit_price NUMERIC(12, 2) NOT NULL
);

-- PERFORMANCE INDEXES
CREATE INDEX idx_users_dealer ON users(dealer_id);
CREATE INDEX idx_products_dealer_sku ON products(dealer_id, sku);
CREATE INDEX idx_inventory_wh_prod ON inventory(warehouse_id, product_id);
CREATE INDEX idx_stock_movements_dealer_prod ON stock_movements(dealer_id, product_id);
CREATE INDEX idx_sales_orders_dealer_status ON sales_orders(dealer_id, status);
```

4. Core Business Workflows & Execution Mechanics

4.1 Inventory Math & Real-Time Stock Formula

To eliminate stockoverselling, physical inventory is separated into two distinct state variables:

- `quantity_on_hand`: Physical units sitting inside the building shelves.
- `quantity_allocated`: Units reserved for approved Sales Orders waiting to be packed/shipped.
- **Available Stock Formula:** $$\textit{Available Stock} = \textit{quantity\_on\_hand} - \textit{quantity\_allocated}$$

4.2 Inbound Procurement Cycle (Manufacturer → Warehouse)

1. **PO Creation:** Dealer Manager creates PO for Manufacturer specifying products, bulk quantities, and factory cost. Status = `DRAFT` or `ORDERED`.
2. **Goods Receiving (GRN):** When truck arrives at warehouse, Staff logs received quantity against PO items.
3. **Ledger & Stock Increment:** System atomically increases `quantity_on_hand` and inserts a `PO_RECEIPT` record into `stock_movements`. If received quantity matches ordered, PO status becomes `COMPLETED`; otherwise `PARTIAL`.

4.3 Outbound Fulfillment Cycle (Warehouse → Shop)

1. **SO Placement:** Manager creates SO for a Shop. System verifies that $$(\textit{Shop Balance} + \textit{SO Total}) \leq \textit{Credit Limit}$$.
2. **Stock Allocation Lock:** Upon approval, system locks stock by increasing `quantity_allocated` by ordered quantity. Available stock drops immediately.
3. **Pick & Pack:** Warehouse Staff generates Pick List, gathers goods, packs them into cartons, and sets status to `PICKING` → `SHIPPED`.
4. **Delivery & Settlement:** Driver delivers goods to shop. On confirmation (`DELIVERED` state), system atomically:
   - Decrements `quantity_on_hand` by delivered count.
   - Decrements `quantity_allocated` by delivered count.
   - Increments Shop's `current_balance` by invoice total.
   - Inserts `SO_DISPATCH` movement into `stock_movements`.

5. REST API Endpoint Contracts & Payloads

HTTP Method	Endpoint Route	Min Role	Operation Description
POST	/api/v1/auth/register-dealer	Public	Executes atomic registration transaction (Dealer + Admin User + Default WH).
POST	/api/v1/auth/login	Public	Validates credentials, returns JWT token with embedded tenant context.
GET	/api/v1/inventory	Staff	Lists live inventory levels, allocated stock, and available quantities.
POST	/api/v1/purchase-orders	Manager	Issues a new Purchase Order to a Manufacturer.
POST	/api/v1/purchase-orders/:id/grn	Staff	Logs Goods Receipt Note (GRN), updates `quantity_on_hand` and movement ledger.
POST	/api/v1/sales-orders	Manager	Creates Sales Order, checks shop credit limit, locks stock allocation.
PATCH	/api/v1/sales-orders/:id/deliver	Driver	Marks order delivered, settles shop balance, decrements physical stock.

Sample Payload: Sales Order Creation DTO

```
POST /api/v1/sales-orders
Authorization: Bearer
Content-Type: application/json

{
  "shopId": "d71842e1-8910-4f91-8192-31049281a100",
  "warehouseId": "b201948d-6712-4211-901a-8812736102aa",
  "items": [
    {
      "productId": "p1102938-1234-5678-9012-112233445566",
      "quantity": 100,
      "unitPrice": 12.50
    }
  ]
}
```

6. NestJS Production Code Blueprints

6.1 Tenant Context Interceptor (`tenant.interceptor.ts`)

```
import { Injectable, NestInterceptor, ExecutionContext, CallHandler, UnauthorizedException } from '@nestjs/
common';
import { Observable } from 'rxjs';

@Injectable()
export class TenantInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable {
    const request = context.switchToHttp().getRequest();
    const user = request.user; // Set by JwtAuthGuard

    if (!user || !user.dealerId) {
      throw new UnauthorizedException('Tenant context missing from JWT token');
    }

    // Attach tenantId directly to request object for easy service access
    request.dealerId = user.dealerId;
    return next.handle();
  }
}
```

6.2 Concurrency-Safe Order Allocation Service (`fulfillment.service.ts`)

```
import { Injectable, BadRequestException } from '@nestjs/common';
import { PrismaService } from '../prisma/prisma.service';

@Injectable()
export class FulfillmentService {
  constructor(private prisma: PrismaService) {}

  async createAndAllocateSalesOrder(dealerId: string, dto: CreateSalesOrderDto) {
    return await this.prisma.$transaction(async (tx) => {
      // 1. Verify Shop Credit Limit
      const shop = await tx.shop.findFirst({
        where: { id: dto.shopId, dealerId }
      });
      if (!shop) throw new BadRequestException('Shop not found');

      const totalAmount = dto.items.reduce((sum, item) => sum + (item.quantity * item.unitPrice), 0);
      if (Number(shop.currentBalance) + totalAmount > Number(shop.creditLimit)) {
        throw new BadRequestException('Order exceeds shop credit limit');
      }

      // 2. Lock Inventory Rows for Update (Pessimistic Locking)
      for (const item of dto.items) {
        const [inventory] = await tx.$queryRaw`
          SELECT * FROM inventory
          WHERE warehouse_id = ${dto.warehouseId}::uuid
            AND product_id = ${item.productId}::uuid
          FOR UPDATE
        `;

        if (!inventory) {
          throw new BadRequestException(`Product ${item.productId} not stocked in warehouse`);
        }

        const available = inventory.quantity_on_hand - inventory.quantity_allocated;
        if (available < item.quantity) {
          throw new BadRequestException(`Insufficient stock for product ${item.productId}. Available: $
{available}`);
        }

        // Increment quantity_allocated
        await tx.inventory.update({
          where: { id: inventory.id },
          data: { quantityAllocated: { increment: item.quantity } }
        });
      }

      // 3. Create Sales Order Record
      const salesOrder = await tx.salesOrder.create({
        data: {
          dealerId,
          shopId: dto.shopId,
          warehouseId: dto.warehouseId,
          soNumber: `SO-${Date.now()}`,
          status: 'APPROVED',
          totalAmount,
          items: {
            create: dto.items.map(i => ({
              productId: i.productId,
              quantityOrdered: i.quantity,
              unitPrice: i.unitPrice
            }))
          }
        }
      });

      return salesOrder;
    });
```

```
  }
}
```

7. Phase 1 Implementation Roadmap & Sprint Plan

Sprint #	Target Module	Core Deliverables & Sprint Goals
Sprint 1	IAM & Tenant Setup	• Setup NestJS + PostgreSQL + Prisma ORM infrastructure. • Implement Auth module (Login, Argon2id password hashing, JWT issue). • Create atomic dealer registration transaction endpoint. • Build Tenant Interceptor and RBAC guards.
Sprint 2	Catalog & Warehouses	• Build CRUD APIs for Warehouses, Categories, and Products (SKUs/Barcodes). • Build CRM modules for Manufacturers and Shops (Credit limit tracking). • Implement React Frontend layout with global Zustand Auth state.
Sprint 3	Inbound Procurement	• Build Purchase Order issuance workflow. • Build Goods Receipt Note (GRN) receiving interface for warehouse staff. • Implement stock increment logic and `stock_movements` ledger creation.
Sprint 4	Outbound Fulfillment	• Implement Sales Order creation with Credit Limit validation. • Add `SELECT ... FOR UPDATE` pessimistic stock allocation locking. • Build Pick & Pack list generation view for warehouse staff. • Build Driver delivery slip & confirmation API.
Sprint 5	Testing & Deployment	• Execute load and concurrency stress testing on stock allocation. • Perform security audit (IDOR verification on all endpoints). • Containerize application with Docker & setup CI/CD pipeline.